Foundations of AI **Fall 2026**
Professor Amy Greenwald **Search Unit**

Blind Search

The first topic of this course is **search**. Search algorithms are used to solve certain types of **search** (alternatively, **optimization**) problems. We all encounter search problems every day. For example, the first day of class, you probably searched for the best path from your dorm to this classroom. This problem of finding the *best* path from your dorm to this classroom is a search problem.

Other examples of search problems include:

- *Rubik's cube*:

- *Knight's tour*: find a sequence of moves on a chess board by which a knight can move in its L-shaped pattern so that it lands on each and every square exactly once
- *Course scheduling*: given degree requirements, course offerings and times, and student preferences, find a(n optimal) course schedule for the semester
- *Game playing*: given the rules of game (e.g., chess), find an optimal policy, meaning an optimal mapping from states to moves
- *Theorem proving*: given a set of axioms, some rules of inference, and a select formula, find a proof of the formula, if one exists

In the first month or so of this course, we will study various flavors of search, including:

1. **Blind search**: depth-first search (DFS), breadth-first search (BFS), iterative deepening search (IDS)
2. **Heuristic search**: in which search is guided by a heuristic evaluation function: A* and IDA*
3. **Adversarial search**: for playing games with mutliple players: minimax and $\alpha\beta$ pruning
4. **Local search**: (stochastic) gradient descent, or hill climbing, and simulated annealing

1 Shakey the Robot, Revisited

Recall our friend Shakey from last lecture, the first mobile robot (pictured here again to jog your memory). Shakey was tasked with things like pushing objects into specific locations and clearing roadblocks. To succeed, Shakey had to be a competent navigator, so that it could find said objects and move itself to them.

Fast forward 50 years. Today's self-driving cars are tasked with a similar navigation problem, namely moving themselves to a specified destination. It does not matter whether an agent is navigating indoors, on roads, or on sidewalks. From an abstract point of view (i.e., from 1000 feet), these problems are one and the same!

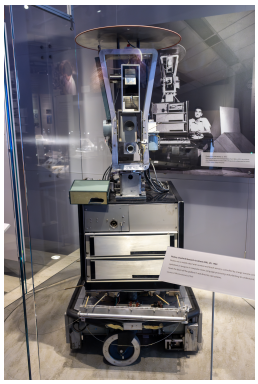


Figure 1: Shakey the Robot. Image source: Wikipedia.

At an abstract level, all navigation problems contains the same components: an environment, a starting location, a goal location, and rules that guide how an agent can move through its environment.

Because these specific navigation problems are the same abstract search problem, if we develop algorithms that solve abstract search problems, these algorithms will apply to all such problems.

N.B. One big difference between Shakey and today's self-driving cars is that the obstacles in Shakey's environment were static, whereas self-driving cars must circumvent moving obstacles with "minds of their own:" i.e., other autonomous agents![1]

2 Basic Search Problem

The first class of problems we encounter in this course is that of basic search problems. A *basic search problem* is a 4-tuple $\langle X, S, G, \mathcal{T} \rangle$, where

- X is a set of states
- $S \subseteq X$ is a nonempty set of *start* states
- $G \subseteq X$ is a nonempty set of *goal* states
- $\mathcal{T} : X \rightrightarrows X$ is a state transition model, mapping a state $x \in X$ to a set $\mathcal{T}(x)$ of successor states

Navigation as Search Given this general problem definition, we can formulate the problem of navigating (i.e., path planning) around Brown's campus as follows:

- X: the set of all intersections on a campus map
- S: the origin of your journey
- G: your desired destination
- $\mathcal{T}$: a mapping from a state to all states "reachable" from that state (e.g., intersections one block away, accounting for dead ends and one-way streets).

[1] Professor Greenwald's research lab studies precisely this problem: how to design autonomous agents in the presence of other autonomous agents, i.e., in multiagent systems, where other agents have "minds of their own."

The **solution** to a basic search problem is a *path* $P = \{x_1, \ldots, x_k\}$, meaning a sequence of states, beginning at some start state $x_1 \in S$ and ending at some goal state $x_k \in G$, with $x_{i+1} \in T(x_i)$ for all $i \in \{2, \ldots, k-1\}$. (**N.B.** Sometimes we are only interested in the goal state, and not the path to the goal state.)

Note that there may be many solutions to a basic search problem: i.e., many paths from a start state to a goal. For example, there are multiple routes from your dorm room to this classroom. Often, we are interested in optimal solutions. An **optimal** solution to a basic search problem is a shortest path, meaning one that traverses the fewest number of states en route from the start state to a goal. In the context of navigation, an optimal path is a **shortest** path: i.e., one with the fewest number of hops.

Knuth's Conjecture In 1964, Donald Knuth conjectured that, starting from the number 4, you can reach any positive integer by applying a sequence of factorial, square root, and floor operators [1].

Here's one way to reach 5:

$$\lfloor (\sqrt{\sqrt{\sqrt{\sqrt{\sqrt{(4!)!}}}}} \rfloor = 5$$

Exercise How would you formulate Knuth's conjecture as search?

- X: the set of positive real numbers ($\mathbb{R}^+$)
- S: 4
- G: $\{5, 6, 7, \ldots\}$
- $\mathcal{T}$: $x \mapsto \{x!, \sqrt{x}, \lfloor x \rfloor\}$

Assumptions Without proof one way or the other, we cannot assume the state space in Knuth's search problem is finite. In these notes, we restrict our attention to search problems on **locally finite** graphs. In locally finite graphs, the degree of every vertex is finite (i.e., no vertex has an infinite number of neighbors).

That said, X itself need *not* be finite. X may be infinite, and even when X is finite, there may be search paths of infinite length. (Imagine Shakey can move north, south, east, and west. Moving east and then west and then east again, repeatedly, would constitute an infinite path, even in a finite state space.)

Since X can, in general, be very large; we cannot usually store all of X in memory. Consider, for example, searching for a solution to Rubik's cube. On the contrary, we conduct search by visiting states one by one, and keeping track of the set of states we are currently searching (the **open** set; also called the **fringe**), and, when space allows, which states we have already searched (the **closed** set).

Abstraction Although navigation and Knuth's conjecture are very different problems, we have formalized them both as basic search problems. In this sense, they are actually both the *same* problem!

One of the central themes of this course is problem formulation. More often than not, the challenge of solving a problem lies in formulating it.

In this week's homework, you will solve mazes, which are perhaps obviously search problems: the goal is to find a path from the start to the finish. You will also formulate a second problem as search, namely maze generation: i.e., the problem of generating a maze starting from a grid of cells with walls surrounding every one by erasing walls until there is (preferably just one) path from the start to the finish.

What is the advantage of abstraction? It saves you work! The alternative is to develop algorithms to solve Rubik's cube; and other algorithms to solve Sokoban; and yet more algorithms to solve Knight's tour; and so on. Alternatively, you can develop a suite of algorithms to solve search problems, and then if you can formulate the problem at hand as search, all of your algorithms (e.g., BFS, DFS, etc.) immediately apply!

3 Generic Search Algorithm

The blind search algorithms we discuss in this lecture are all instantiations of the following generic search algorithm. This algorithm maintains an open set of states, initialized to the set of start states, which it considers in turn as potential goal states. For each such state, it either deems it a goal and returns, or it discards it and inserts its successors into the open set to also be considered in turn as potential goal states.

When a state is deleted from the open set and tested to see if it is a goal or not, we say that state has been **visited**. When a state's successors are added to the open set, we say that state has been **expanded**.

SEARCH	
Inputs	search problem $\langle X, S, G, \mathcal{T} \rangle$
Output	(path to) goal node
Initialize	$O = S$ is the list of open nodes

`while` (O is not empty) `do`

1. delete some node $n \in O$
2. if $n \in G$, return (path to) n
3. insert $\mathcal{T}(n)$ into O

`fail`

Table 1: Generic Search Algorithm.

This algorithm is designed for trees, not graphs. It does not maintain a closed set, because there is exactly one path to every node in a tree.

Exercise Modify this algorithm so that it searches graphs, not just trees, by keeping track of a closed set as well as an open one and making sure not to revisit already visited states. You can assume access to sufficient memory in which to store all visited states, and sufficient time to test membership in this set.

4 Evaluation Criteria

The following criteria are used to evaluate search algorithms:

- time complexity: how much time is required to find solution?
- space complexity: how much space is required to find solution?
- completeness: is the algorithm guaranteed to find solution, if one exists?
- optimality: does it find an optimal solution?

The analyses and algorithms presented in this lecture depend on the assumption that the search space is a tree of (possibly infinite) depth d with finite branching factor b. The **depth** of a tree is defined as the maximum number of hops from the root node to any other node in the tree. The **branching factor** of a tree is defined as the maximum number of children of any node in the tree.

5 Breadth-First Search

The main idea of breadth-first search (BFS) is to visit all the states at depth i before visiting those at depth $i+1$. BFS is implemented by storing the open set as a *queue*, and accessing its entries in a first-in-first-out (FIFO) fashion.

BFS	
Inputs	search problem $\langle X, S, G, \mathcal{T} \rangle$
Output	(path to) goal node
Initialize	$O = S$ is the list of open nodes

`while` (O is not empty) `do`

1. delete *first* node $n \in O$
2. if $n \in G$, return (path to) n
3. append $\mathcal{T}(n)$ to *back* of O

`fail`

Table 2: Breadth-First Search.

BFS is complete: it is guaranteed to find a solution, if one exists. Moreover, BFS is optimal: it always finds a goal state of minimal distance from the start state.

In the worst case, BFS expands every node, which takes time as follows:

$$1 + b + b^2 + \ldots + b^d = \sum_{i=0}^{d} b^i = \frac{b^{d+1} - 1}{b - 1} = O(b^d)$$

In terms of space, BFS maintains a queue of potentially all the states at each depth: at depth d, the length of this queue is b^d. The space complexity of BFS is also $O(b^d)$.

Thus, both the time and space complexities of BFS are exponential in d.

6 Depth-First Search

The main idea of depth-first search (DFS) is to always visit a state of maximal depth, among all open states. DFS is implemented by storing the open set as a *stack*, and accessing its entries in a last-in-first-out fashion (LIFO).

Like BFS, the time complexity of DFS is $O(b^d)$. It is exponential in d because, like BFS, DFS searches the entire tree in the worst case. The space complexity of DFS, however, is linear in d: at most b nodes are stored at each of the d depths: i.e., DFS is $O(bd)$.

DFS$(X, S, G, \mathcal{T})$	
Inputs	search problem
Output	(path to) goal node
Initialize	$O = S$ is the list of open nodes

`while` (O is not empty) `do`

1. delete *first* node $n \in O$
2. if $n \in G$, return (path to) n
3. prepend $\mathcal{T}(n)$ to *front* of O

`fail`

Table 3: Depth-First Search.

DFS is neither complete nor optimal. What if we were to try using DFS to solve Knuth's conjecture, and it applied the factorial operation repeatedly? The state value would only ever increase and we'd never reach our goal! (Incomplete) What about a robot who is planning a path to move 1 meter to the east? If DFS started searching west first, the would robot travel all the way around the world before reaching its goal! (Suboptimal)

7 Iterative Deepening

Iterative deepening (ID) is an optimal search algorithm with the space requirements of DFS—it requires memory linear in d—and the performance properties of BFS—it is complete and optimal. The main idea of iterative deepening is to repeatedly search in depth-first fashion, over subgraphs of depth 0, depth 1, depth 2, and so on, until a goal is found.

ID$(X, S, G, \mathcal{T})$	
Inputs	search problem
Output	(path to) goal node
Initialize	$c = 0$ is the cutoff depth
	$O = S$ is the list of open nodes

`while` *goal not found* `do`

1. `while` (O is not empty) `do`
   (a) delete *first* node $n \in O$
   (b) if $n \in G$, return (path to) goal n
   (c) if depth$(n) \leq c$
      i. prepend $\mathcal{T}(n)$ to *front* of O
2. increment c, $O = S$

Table 4: Iterative Deepening.

ID is optimal and complete: it is guaranteed to find a goal if one exists, and an optimal goal when there are multiple solutions.

Like DFS and BFS, the time complexity of ID is $O(b^d)$: in the worst case it is exponential in d. ID expands nodes at depth 0 $d+1$ times, at depth 1 d times, ..., and at depth d 1 time. Thus, the total time required is given by:

$$1 + \underbrace{O(b)}_{\text{depth 1}} + \underbrace{O(b^2)}_{\text{depth 2}} + \ldots + \underbrace{O(b^d)}_{\text{depth } d} = O(b^d)$$

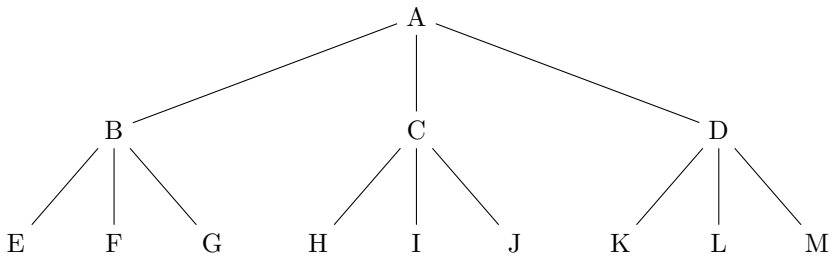
Finally, ID iteratively performs depth-first searches; thus, its space complexity is that of DFS, namely $O(bd)$.

8 Summary

Criteria	DFS	BFS	ID
Time	$O(b^d)$	$O(b^d)$	$O(b^d)$
Space	$O(bd)$	$O(b^d)$	$O(bd)$
Completeness	NO	YES	YES
Optimality	NO	YES	YES

1. BFS is preferred when the branching factor is small
2. ID is preferred when the depth is large: the search space is deep (possibly infinite), particularly if goals are known to be shallow
3. DFS is preferred when the maximum depth of the goal nodes is known: if this depth is n, modify DFS to search only to depth n

9 Example



BFS visits nodes in alphabetical order ABCDEFGHIJKLM.

DFS visits nodes in this order: ABEFGCHIJDKLM.

ID visits nodes in this order AABCDABEFGCHIJDKLM.

References

[1] Donald E. Knuth. Representing numbers using only one 4. *Mathematics Magazine*, 37(5):308–310, 1964.